Introduction

Data warehousing is the process of collecting, storing and managing large amounts of data from different sources to support business analysis and decision-making. A data warehouse provides a central location where historical and current data can be stored, integrated and analysed.

FreshCart is a retail chain that operates 120 supermarkets across six states. It collects data from sales, customers, products, stores, employees, promotions, suppliers and returns. However, its existing operational database is designed mainly for daily transactions rather than complex analytical queries.

To solve this problem, a centralized data warehouse is proposed. It integrates data from different sources using ETL (Extract, Transform and Load), organizes the data using dimensional modelling and provides analytical reports through OLAP operations.

The proposed system helps management understand sales performance, customer behaviour, product demand, promotional effectiveness and store performance.

Objectives

1. To analyse the limitations of the existing operational database.
2. To design a suitable data warehouse architecture.
3. To create star and snowflake dimensional models.
4. To develop an effective ETL process.
5. To perform OLAP operations using data cubes.
6. To evaluate query processing performance.
7. To develop reports using KNIME.
8. To recommend a scalable warehouse design.

TASK 1 – Analyse the Decision-Support Requirement (4 Marks)

1.1 Limitations of the Existing Operational System

FreshCart currently uses an OLTP (Online Transaction Processing) database to manage daily activities such as billing, orders, customer details, products and returns.

The operational database is suitable for inserting, updating and deleting transactions. However, it is not suitable for performing complex management analysis.

The major problems are:

1. Slow queries: Analytical queries involving millions of sales records can slow down billing operations.

2. Data duplication: Customer and product information may be repeated across different transaction records.
3. Inconsistent reports: Different departments calculate net sales differently.
4. Limited historical analysis: OLTP databases mainly maintain current operational information.
5. Complex joins: Analytical reports require joining several normalized tables.
6. Performance issues: Running multiple analytical queries affects daily business operations.
7. Limited decision-making: It is difficult to identify long-term sales trends and customer behaviour.
8. Data integration problems: Data from physical stores and online channels must be combined.

Therefore, FreshCart requires a separate Data Warehouse and Decision-Support System.

1.2 Business Questions, Measures and Dimensions

Business questions are classified into two categories:

- Strategic questions: Used by top-level management for long-term planning.
- Tactical questions: Used by department managers for short-term operational decisions.

No.	Business question	Type	Measures	Dimensions
1	Which product categories show declining net sales quarter over quarter?	Strategic	Net sales	Time, Product, Region
2	Which promotions increase average basket value?	Strategic	Sales amount, Basket value	Promotion, Store, Time
3	How do weekend sales differ from weekday sales?	Tactical	Total sales, Quantity	Time, Store, Department
4	Which customer segments return the most items?	Tactical	Return quantity, Return amount	Customer, Product, Time
5	What is the year-to-date sales of every store compared with last year?	Strategic	Current sales, Previous sales	Store, Time
6	Which products generate the highest revenue?	Tactical	Net sales, Profit	Product, Store, Time

7	Which regions contribute the highest sales?	Strategic	Total sales	Region, Time
8	Which stores have the highest average transaction value?	Tactical	Average basket value	Store, Time

1.3 Operational System vs Decision-Support System

Feature	OLTP System	Decision-Support System
Purpose	Daily transactions	Business analysis
Data	Current data	Historical and current data
Structure	Normalized tables	Dimensional models
Queries	Simple transactions	Complex analytical queries
Users	Cashiers and employees	Managers and analysts
Operations	Insert, update, delete	Read, analyse and summarize
Volume	Individual transactions	Large volumes of integrated data
Performance	Fast transaction processing	Fast analytical processing

Application to FreshCart

For example, a cashier needs to retrieve the price of a product immediately during billing. This is an OLTP operation.

However, management may want to compare the sales of all product categories across six states for the previous three years. This requires joining and aggregating a large amount of historical data.

Performing this analysis directly on the operational database may slow down billing counters.

Therefore, FreshCart should use OLTP for daily transactions and a separate data warehouse for analytical processing.

1.4 Conclusion

A separate decision-support system is necessary for FreshCart because it improves analytical performance, maintains historical information, provides consistent reports and supports better business decisions.

TASK 2 – Propose the Warehouse Architecture (3 Marks)

2.1 Proposed Data Warehouse Architecture

FreshCart requires an Enterprise Data Warehouse (EDW) that collects information from all its supermarkets and online sales systems.

The architecture consists of the following components:

1. Data Sources

Data is collected from:

- Orders and OrderLines
- Customers
- Products
- Stores
- Employees
- Promotions
- Suppliers
- Returns
- Online sales system

2. Staging Area

The staging area temporarily stores extracted data before loading it into the warehouse. It helps identify missing values, duplicate records and inconsistent formats.

3. ETL Process

ETL consists of three stages:

- Extract: Collect data from source databases.
- Transform: Clean, validate and standardize the data.
- Load: Store the processed data in the warehouse.

4. Data Warehouse

The warehouse stores integrated historical data in fact and dimension tables.

5. Data Marts

Separate data marts can be created for:

- Sales analysis
- Customer analysis
- Product analysis
- Store performance

6. OLAP Server

The OLAP server performs analytical operations such as roll-up, drill-down, slice, dice and pivot.

7. Front-End Tools

Management can access reports and dashboards using:

- Metabase
- Apache Superset
- Pentaho
- KNIME

2.2 Architecture Diagram

Figure 2.1: Proposed Data Warehouse Architecture

2.3 Selected Architecture

FreshCart should use an Enterprise Data Warehouse with dependent data marts.

Reasons:

1. It integrates data from all 120 supermarkets.
2. It provides a centralized source of information.
3. It maintains consistent sales calculations.
4. It supports both physical and online sales.
5. Dependent data marts provide department-specific analysis.
6. It can be expanded when the number of transactions increases.

2.4 Conclusion

The proposed architecture provides centralized data storage, efficient ETL processing and faster analytical reporting. It is suitable for FreshCart's current and future requirements.

TASK 3 – Design the Star Schema (6 Marks)

3.1 Business Process

The selected business process is Retail Sales Analysis.

The system analyses sales transactions from all FreshCart supermarkets.

3.2 Grain of the Fact Table

The grain of the fact table is:

One row represents one product line item sold in a particular order at a particular store on a particular date, for a particular customer and promotion.

Defining the grain is important because every measure and dimension must represent the same level of detail.

3.3 Fact Table – Fact_Sales

The fact table stores numerical information about sales transactions.

Attribute	Description
Sales_Key	Primary key
Date_Key	Foreign key
Product_Key	Foreign key
Store_Key	Foreign key
Customer_Key	Foreign key
Promotion_Key	Foreign key
Order_ID	Transaction identifier
Quantity	Number of products sold
Unit_Price	Price per product
Gross_Sales	Quantity × Unit Price
Discount	Discount amount
Net_Sales	Gross Sales − Discount
Cost_Amount	Total product cost
Profit	Net Sales − Cost Amount

Classification of Measures

Measure	Type	Explanation
Quantity	Additive	Can be summed across dimensions
Gross_Sales	Additive	Can be summed
Discount	Additive	Can be summed
Net_Sales	Additive	Can be summed
Cost_Amount	Additive	Can be summed

Profit	Additive	Can be summed
Unit_Price	Non-additive	Should not be directly summed
Profit Margin	Non-additive	Must be recalculated using totals

An average selling price should be calculated as total sales divided by total quantity, not by adding individual prices.

3.4 Dimension Tables

1. Dim_Date

Attribute	**Description**
Date_Key	Primary key
Full_Date	Complete date
Day	Day number
Month	Month
Quarter	Quarter
Year	Year
Weekday	Weekday name
Weekend_Flag	Weekend or weekday

Hierarchy:

Day → Month → Quarter → Year

2. Dim_Product

Attribute	**Description**
Product_Key	Primary key
Product_ID	Business identifier
Product_Name	Product name
Brand	Brand name
Category	Product category
Department	Department

Unit	Product measurement unit

Hierarchy:

Product → Category → Department

3. Dim_Store

Attribute	Description
Store_Key	Primary key
Store_ID	Business identifier
Store_Name	Store name
City	City
State	State
Region	Region

Hierarchy:

Store → City → State → Region

4. Dim_Customer

Attribute	Description
Customer_Key	Primary key
Customer_ID	Business identifier
Customer_Name	Customer name
Gender	Gender
Age_Group	Age category
Customer_Segment	Customer classification
Address	Customer address
Loyalty_Tier	Loyalty category

5. Dim_Promotion

Attribute	Description
Promotion_Key	Primary key

Promotion_ID	Business identifier
Promotion_Name	Promotion name
Promotion_Type	Promotion category
Discount_Percentage	Discount percentage
Start_Date	Starting date
End_Date	Ending date

3.5 Star Schema Diagram

Figure 3.1: FreshCart Star Schema

The Fact_Sales table is placed at the centre, and all five dimension tables are directly connected to it using foreign keys.

3.6 Primary Keys, Surrogate Keys and Foreign Keys

- Primary Key: Uniquely identifies each record in a table.
- Surrogate Key: Artificial key generated by the warehouse, such as Product_Key.
- Foreign Key: Connects a fact table with its dimension table.

Surrogate keys are preferred because business identifiers can change between source systems.

3.7 Second Fact Table – Fact_Returns

FreshCart also requires a separate fact table for analysing returned products.

Attribute	**Description**
Return_Key	Primary key
Date_Key	Foreign key
Product_Key	Foreign key
Store_Key	Foreign key
Customer_Key	Foreign key
Return_ID	Return identifier
Return_Quantity	Number of returned products
Return_Amount	Total refund amount

Both Fact_Sales and Fact_Returns use the same Date, Product, Store and Customer dimensions. These are called conformed dimensions.

3.8 SQL Table Creation Statements

The following SQL uses standard relational database syntax.

```
CREATE TABLE Dim_Date (
    Date_Key INT PRIMARY KEY,
    Full_Date DATE,
    Day INT,
    Month INT,
    Quarter INT,
    Year INT,
    Weekday VARCHAR(20),
    Weekend_Flag VARCHAR(5)
);

CREATE TABLE Dim_Product (
    Product_Key INT PRIMARY KEY,
    Product_ID VARCHAR(20),
    Product_Name VARCHAR(100),
    Brand VARCHAR(50),
    Category VARCHAR(50),
    Department VARCHAR(50)
);

CREATE TABLE Dim_Store (
    Store_Key INT PRIMARY KEY,
    Store_ID VARCHAR(20),
    Store_Name VARCHAR(100),
    City VARCHAR(50),
    State VARCHAR(50),
    Region VARCHAR(50)
);

CREATE TABLE Dim_Customer (
    Customer_Key INT PRIMARY KEY,
    Customer_ID VARCHAR(20),
    Customer_Name VARCHAR(100),
    Customer_Segment VARCHAR(50),
    Address VARCHAR(200),
    Loyalty_Tier VARCHAR(30)
);

CREATE TABLE Dim_Promotion (
    Promotion_Key INT PRIMARY KEY,
```

```
    Promotion_ID VARCHAR(20),
    Promotion_Name VARCHAR(100),
    Promotion_Type VARCHAR(50),
    Discount_Percentage DECIMAL(5,2)
);

CREATE TABLE Fact_Sales (
    Sales_Key INT PRIMARY KEY,
    Date_Key INT,
    Product_Key INT,
    Store_Key INT,
    Customer_Key INT,
    Promotion_Key INT,
    Order_ID INT,
    Quantity INT,
    Unit_Price DECIMAL(10,2),
    Gross_Sales DECIMAL(12,2),
    Discount DECIMAL(12,2),
    Net_Sales DECIMAL(12,2),
    Cost_Amount DECIMAL(12,2),
    Profit DECIMAL(12,2),

    FOREIGN KEY (Date_Key)
        REFERENCES Dim_Date(Date_Key),

    FOREIGN KEY (Product_Key)
        REFERENCES Dim_Product(Product_Key),

    FOREIGN KEY (Store_Key)
        REFERENCES Dim_Store(Store_Key),

    FOREIGN KEY (Customer_Key)
        REFERENCES Dim_Customer(Customer_Key),

    FOREIGN KEY (Promotion_Key)
        REFERENCES Dim_Promotion(Promotion_Key)
);
```

3.9 Conclusion

The star schema provides a simple and efficient structure for FreshCart. It reduces complex joins and makes sales analysis easier. It also supports historical analysis using different dimensions.

TASK 4 – Compare with a Snowflake Schema (4 Marks)

4.1 Introduction

A snowflake schema is an extension of a star schema in which dimension tables are divided into multiple related tables to reduce data redundancy.

In FreshCart, Product and Store dimensions can be normalized into smaller tables.

4.2 Snowflake Schema Design

Product Dimension

Instead of storing all product information in one table, it is divided into:

1. Dim_Product
2. Dim_Brand
3. Dim_Category
4. Dim_Department

Relationship:

Product → Brand → Category → Department

Store Dimension

The Store dimension is divided into:

1. Dim_Store
2. Dim_City
3. Dim_State
4. Dim_Region

Relationship:

Store → City → State → Region

Figure 4.1: Snowflake Schema for FreshCart

4.3 Additional Tables and Keys

Table	Primary Key	Foreign Key
Dim_Product	Product_Key	Brand_Key
Dim_Brand	Brand_Key	Category_Key
Dim_Category	Category_Key	Department_Key
Dim_Department	Department_Key	—

Dim_Store	Store_Key	City_Key
Dim_City	City_Key	State_Key
Dim_State	State_Key	Region_Key
Dim_Region	Region_Key	—

4.4 Storage Calculation

Given:

- Products = 50,000
- Brands = 800
- Categories = 60
- Departments = 12
- Stores = 120
- Cities = 70
- States = 6
- Regions = 3

Total records in normalized dimension tables:

\[50,000+800+60+12+120+70+6+3 \]

\[=51,071\text{ records} \]

For comparison, assume the denormalized star dimensions store:

- Product records = 50,000
- Store records = 120

Total = 50,120 records.

However, row count alone does not measure storage accurately because snowflake tables contain additional keys and the star schema repeats descriptive attributes.

For an illustrative storage estimate, assume the following:

Dimension	Star storage assumption
Product descriptions and hierarchy attributes	160 bytes per product
Store descriptions and hierarchy attributes	100 bytes per store

Star schema:

$$ (50,000 \times 160)+(120 \times 100) $$

$$ =8,012,000\text{ bytes} $$

For the snowflake schema, assume product rows require 70 bytes, brand rows 50 bytes, category rows 40 bytes, department rows 30 bytes, store rows 50 bytes, city rows 40 bytes, state rows 30 bytes and region rows 20 bytes.

$$ \begin{aligned} S={}&(50,000\times70)+(800\times50)\\ &+(60\times40)+(12\times30)\\ &+(120\times50)+(70\times40)\\ &+(6\times30)+(3\times20) \end{aligned} $$

$$ S=3,579,810\text{ bytes} $$

Estimated storage reduction:

$$ \frac{8,012,000-3,579,810}{8,012,000}\times100 $$

$$ =55.33\% $$

Note: These are illustrative estimates based on assumed attribute widths, excluding indexes, page overhead and database-specific storage overhead. Actual results must be measured.

4.5 Join Comparison

Analytical query	Star schema	Snowflake schema
Sales by product category	1 join	3 joins
Sales by department	1 join	4 joins
Sales by store state	1 join	3 joins

The star schema requires fewer joins because the hierarchy attributes are stored directly in dimension tables.

4.6 Performance Comparison

Feature	Star Schema	Snowflake Schema
Storage	More redundant data	Less repeated descriptive data
Query speed	Generally faster	May be slower due to joins
ETL	Easier	More complex
Maintenance	Simpler	More tables to maintain
Business understanding	Easy	More complex
Joins	Fewer	More

4.7 Recommended Schema

For FreshCart, the star schema is recommended because management frequently needs reports involving product categories, departments, regions and stores.

The star schema offers:

- Faster analytical queries.
- Simple reporting.
- Fewer joins.
- Easier integration with BI tools.

A snowflake schema may be preferred if dimension redundancy becomes a major storage or maintenance concern.

4.8 Conclusion

Both schemas have advantages. However, the star schema is more appropriate for FreshCart because query simplicity and reporting performance are more important than reducing the relatively small dimension storage.

TASK 5 – Design the ETL Process (5 Marks)

5.1 Introduction

ETL stands for Extract, Transform and Load. It is used to collect data from different sources, clean the data and store it in a data warehouse.

FreshCart receives data from multiple supermarkets. Therefore, the ETL process must handle inconsistent formats, missing values, duplicate information and invalid records.

5.2 ETL Architecture

Figure 5.1: ETL Process

5.3 Data Quality Problems and Transformation Rules

The given source data contains six records.

Order ID	Problem	Transformation
1001	Normal record	Accept
1002	Different customer and product naming formats	Standardize and validate customer identity
1003	Negative quantity	Move to return processing
1004	Missing store	Assign Unknown Store key and investigate

1005	Price in USD	Convert using approved exchange rate or reject temporarily
1006	Missing product	Assign Unknown Product key and investigate

Explanation of Each Record

Record 1001: The record is valid. It can be loaded after standard date and numeric validation.

Record 1002: The product name has different capitalization and spacing. It should be standardized to Basmati Rice 5kg. The customer name R. Kumar should be mapped to Rajesh Kumar only after checking a reliable customer identifier.

Record 1003: The quantity is -3. Negative quantity should not be treated as normal sales. It must be routed to the Returns fact table after checking the return transaction.

Record 1004: The store is missing. The record should be assigned an Unknown Store surrogate key and placed in a data-quality review queue.

Record 1005: The unit price is expressed in USD. It must be converted into INR using the approved exchange rate applicable to the transaction date. If the exchange rate is unavailable, the record should be rejected temporarily.

Record 1006: The product value is missing. The record should be assigned an Unknown Product key until the product information is received.

5.4 Cleaned Data

Assume the missing store and product values are handled using unknown dimension members. The USD record remains pending until its exchange rate is verified.

Order ID	Customer	Store	Product	Qty	Unit Price	Order Date	Status
1001	Rajesh Kumar	Chennai-04	Basmati Rice 5kg	2	₹620.00	2026-11-03	Accepted
1002	R. Kumar	Chennai-04	Basmati Rice 5kg	1	₹620.00	2026-11-03	Accepted after validation
1003	Anita Joseph	Pune-11	Olive Oil 1L	-3	₹780.00	2026-11-04	Returns processing
1004	Meena S	Unknown	Tea 250g	4	₹95.50	2026-11-04	Accepted with warning
1005	Anita Joseph	Pune-11	Olive Oil 1L	3	Pending	2026-11-04	Rejected temporarily

1006	Vikram Rao	Hyd-02	Unknown	1	₹1,250.00	2026-11-05	Accepted with warning

The accepted records should retain their original source values and audit information so corrections can be traced later.

5.5 Surrogate-Key Lookup

A surrogate key is a unique warehouse-generated identifier.

For example:

Product_Key	Product_Name
101	Basmati Rice 5kg
102	Olive Oil 1L
103	Tea 250g
0	Unknown Product

When a new sales record arrives, the ETL process searches for the corresponding Product_Key.

If the product is unavailable, the record uses the Unknown Product key until the correct dimension member arrives.

5.6 Slowly Changing Dimension (SCD)

A Slowly Changing Dimension is used to manage changes in dimension attributes.

FreshCart customers may frequently change their addresses and loyalty tiers.

SCD Type 1: Overwrites the old value. Historical information is not maintained.

SCD Type 2: Creates a new dimension record whenever an important attribute changes.

SCD Type 3: Maintains limited previous and current values in separate columns.

Selected Method: SCD Type 2

FreshCart should use SCD Type 2 for customer address and loyalty tier because management wants to analyse sales using the address and loyalty tier applicable at the time of the order.

Example:

Customer_Key	Customer_ID	Address	Loyalty Tier	Start Date	End Date	Current
101	C001	Chennai	Silver	2025-01-01	2026-05-31	No
205	C001	Coimbatore	Gold	2026-06-01	9999-12-31	Yes

The old record is retained, and a new record is inserted.

Sales facts must reference the appropriate historical Customer_Key based on the transaction date.

5.7 Initial Load and Incremental Load

Initial Load	Incremental Load
Loads historical data for the first time	Loads only new or changed records
Processes large volumes	Processes smaller batches
Takes more time	Takes less time
Builds initial dimensions and facts	Updates dimensions and facts

Loading Order

1. Extract source data.
2. Load data into the staging area.
3. Clean and transform records.
4. Load Date, Product, Store, Customer and Promotion dimensions.
5. Perform surrogate-key lookup.
6. Load Fact_Sales and Fact_Returns.
7. Validate row counts and totals.
8. Generate audit reports.

5.8 Rejected Records and Audit

Rejected records must be stored separately with the following details:

- Source system name.
- Order ID.
- Rejection reason.
- Date and time.
- Error description.
- Correction status.

Row-count auditing should compare the number of extracted, accepted, rejected and loaded records.

For example:

Description	Records
Extracted	6
Accepted for processing	5
Rejected temporarily	1
Final status	6 accounted for

The negative-quantity transaction is accounted for separately through return processing.

5.9 Conclusion

The ETL process improves data quality and ensures that FreshCart maintains accurate and consistent warehouse information. SCD Type 2 helps preserve customer history, while surrogate keys support reliable dimension relationships.

TASK 6 – Model the Data Cube and Apply OLAP Operations (5 Marks)

6.1 Introduction

A data cube is a multidimensional structure used to analyse data from different perspectives.

FreshCart uses four dimensions:

1. Time: Day → Month → Quarter → Year
2. Product: Product → Category → Department
3. Store: Store → City → State
4. Customer Segment: Segment

The cube stores sales measures such as quantity, gross sales, discount and net sales.

6.2 Number of Cuboids

A cuboid represents one combination of aggregation levels across the dimensions.

For hierarchical dimensions, each dimension also has an ALL level.

Time has four hierarchy levels plus ALL:

$$ 4+1=5 $$

Product has three levels plus ALL:

$$ 3+1=4 $$

Store has three levels plus ALL:

$$ 3+1=4 $$

Customer Segment has one level plus ALL:

\[1+1=2 \]

Therefore:

\[5\times4\times4\times2 \]

80 cuboids

Without Hierarchies

If all four dimensions are treated as simple dimensions with only their base level and ALL:

\[2^4=16 \]

Therefore:

- With hierarchies = 80 cuboids.
- Without hierarchies = 16 cuboids.

6.3 Base Cuboid and Apex Cuboid

Base Cuboid

The base cuboid contains the most detailed sales information.

It includes:

- Individual dates.
- Individual products.
- Individual stores.
- Individual customer segments.

Apex Cuboid

The apex cuboid contains the highest level of aggregation.

It contains the overall total sales of FreshCart across all dimensions.

6.4 OLAP Operations

1. Roll-Up

Roll-up summarizes detailed information into a higher level.

Business Question: What are the total sales for each product category?

```
SELECT
   p.Category,
   SUM(f.Net_Sales) AS Total_Sales
FROM Fact_Sales f
JOIN Dim_Product p
```

```
    ON f.Product_Key = p.Product_Key
GROUP BY p.Category;
```

This operation moves from product-level sales to category-level sales.

2. Drill-Down

Drill-down moves from summarized information to detailed information.

Business Question: What are the monthly sales of each store in Chennai?

```
SELECT
    d.Year,
    d.Month,
    s.Store_Name,
    SUM(f.Net_Sales) AS Total_Sales
FROM Fact_Sales f
JOIN Dim_Date d
    ON f.Date_Key = d.Date_Key
JOIN Dim_Store s
    ON f.Store_Key = s.Store_Key
WHERE s.City = 'Chennai'
GROUP BY d.Year, d.Month, s.Store_Name;
```

This operation provides more detailed information about store performance.

3. Slice

Slice selects one fixed value from a dimension.

Business Question: What are the sales of FreshCart during 2026?

```
SELECT
    SUM(f.Net_Sales) AS Total_Sales
FROM Fact_Sales f
JOIN Dim_Date d
    ON f.Date_Key = d.Date_Key
WHERE d.Year = 2026;
```

This query selects one year from the Time dimension.

4. Dice

Dice selects multiple values from multiple dimensions.

Business Question: What are the sales of rice products in Chennai and Pune during 2026?

```
SELECT

    p.Product_Name,
    s.City,
    SUM(f.Net_Sales) AS Total_Sales
FROM Fact_Sales f
```

```
JOIN Dim_Product p
   ON f.Product_Key = p.Product_Key
JOIN Dim_Store s
   ON f.Store_Key = s.Store_Key
JOIN Dim_Date d
   ON f.Date_Key = d.Date_Key
WHERE p.Category = 'Rice'
AND s.City IN ('Chennai', 'Pune')
AND d.Year = 2026
GROUP BY p.Product_Name, s.City;
```

This operation filters multiple dimensions.

5. Pivot

Pivot changes the orientation of the report to compare different dimensions.

Business Question: How do sales of different product categories compare across stores?

```
SELECT

   p.Category,
   s.Store_Name,
   SUM(f.Net_Sales) AS Total_Sales
FROM Fact_Sales f
JOIN Dim_Product p
   ON f.Product_Key = p.Product_Key
JOIN Dim_Store s
   ON f.Store_Key = s.Store_Key
GROUP BY p.Category, s.Store_Name;
```

The result can be displayed as a pivot table with product categories as rows and stores as columns in the reporting application.

6.5 Cuboid Materialization

Materialization means precomputing and storing selected cuboids to improve query performance.

FreshCart should materialize:

1. Year × Region – for strategic management reports.
2. Quarter × Category – for product performance.
3. Month × Store – for monthly store reports.
4. Month × Customer Segment – for customer analysis.

These cuboids are selected because they support frequently requested management reports.

Storing every possible cuboid would require additional storage and maintenance. Therefore, partial materialization provides a balance between performance and storage cost.

6.6 Conclusion

Data cubes and OLAP operations help FreshCart analyse sales from different perspectives. They support faster reporting and help management identify business trends.

TASK 7 – Develop an OLAP Application (5 Marks)

7.1 Introduction

An OLAP application allows users to analyse warehouse data using different dimensions and aggregation levels.

For FreshCart, an OLAP application can be developed using Python, SQL and a database such as SQLite or MySQL.

The application should allow users to select different operations and view the corresponding sales results.

7.2 Technologies Used

Technology	Purpose
Python	Application development
SQLite / MySQL	Warehouse database
Pandas	Data processing
Matplotlib	Graph generation
KNIME	Reporting and visualization

7.3 Application Workflow

Figure 7.1: OLAP Application Workflow

7.4 Working of the Application

The application performs the following steps:

1. Connects to the warehouse database.
2. Retrieves sales records from Fact_Sales.
3. Allows the user to select an OLAP operation.
4. Accepts dimension and filter values.
5. Executes the corresponding SQL query.
6. Displays the result in tabular or graphical format.

7. Allows the user to compare sales information.

7.5 Python Application Example

The following is a simple SQLite-based example. It assumes the tables created in Task 3 have been populated.

```
CREATE TABLE Dim_Date (
    Date_Key INT PRIMARY KEY,
    Full_Date DATE,
    Day INT,
    Month INT,
    Quarter INT,
    Year INT,
    Weekday VARCHAR(20),
    Weekend_Flag VARCHAR(5)
);

CREATE TABLE Dim_Product (
    Product_Key INT PRIMARY KEY,
    Product_ID VARCHAR(20),
    Product_Name VARCHAR(100),
    Brand VARCHAR(50),
    Category VARCHAR(50),
    Department VARCHAR(50)
);

CREATE TABLE Dim_Store (
    Store_Key INT PRIMARY KEY,
    Store_ID VARCHAR(20),
    Store_Name VARCHAR(100),
    City VARCHAR(50),
    State VARCHAR(50),
    Region VARCHAR(50)
);

CREATE TABLE Dim_Customer (
    Customer_Key INT PRIMARY KEY,
    Customer_ID VARCHAR(20),
    Customer_Name VARCHAR(100),
    Customer_Segment VARCHAR(50),
    Address VARCHAR(200),
    Loyalty_Tier VARCHAR(30)
);
```

```
CREATE TABLE Dim_Promotion (
    Promotion_Key INT PRIMARY KEY,
    Promotion_ID VARCHAR(20),
    Promotion_Name VARCHAR(100),
    Promotion_Type VARCHAR(50),
    Discount_Percentage DECIMAL(5,2)
);

CREATE TABLE Fact_Sales (
    Sales_Key INT PRIMARY KEY,
    Date_Key INT,
    Product_Key INT,
    Store_Key INT,
    Customer_Key INT,
    Promotion_Key INT,
    Order_ID INT,
    Quantity INT,
    Unit_Price DECIMAL(10,2),
    Gross_Sales DECIMAL(12,2),
    Discount DECIMAL(12,2),
    Net_Sales DECIMAL(12,2),
    Cost_Amount DECIMAL(12,2),
    Profit DECIMAL(12,2),

    FOREIGN KEY (Date_Key)
        REFERENCES Dim_Date(Date_Key),

    FOREIGN KEY (Product_Key)
        REFERENCES Dim_Product(Product_Key),

    FOREIGN KEY (Store_Key)
        REFERENCES Dim_Store(Store_Key),

    FOREIGN KEY (Customer_Key)
        REFERENCES Dim_Customer(Customer_Key),

    FOREIGN KEY (Promotion_Key)
        REFERENCES Dim_Promotion(Promotion_Key)
);
```

This example demonstrates the five OLAP query patterns. For a complete interactive application, additional controls can be added for selecting hierarchy levels, filters and pivot-table columns.

7.6 Data Generation

FreshCart can generate synthetic sales data to test the warehouse at different sizes.

Dataset	Number of Fact Rows
Small	1,000
Medium	10,000
Large	100,000
Very Large	1,000,000

Synthetic data can be generated using Python's random module.

The generated records should contain valid date, product, store, customer and promotion keys. Random values should be created for quantity, unit price, discounts and cost.

7.7 Result Validation

The OLAP results must be compared with independent calculations from the source data.

For example:

$$\text{Total Net Sales}=\sum \text{Net Sales}$$

The total sales obtained from the warehouse should match the independently calculated source total after accounting for rejected records, returns and approved transformations.

7.8 Conclusion

The OLAP application provides an interactive method to analyse FreshCart sales data. It allows users to perform different analytical operations and understand business performance.

TASK 8 – Evaluate Query Processing Performance (4 Marks)

8.1 Introduction

Query processing performance measures how efficiently a database executes analytical queries.

FreshCart must process increasing numbers of sales records. Therefore, the performance of the OLTP schema and data warehouse must be compared.

Three database configurations are considered:

1. Normalized OLTP schema.
2. Star schema.
3. Star schema with indexes and aggregate tables.

8.2 Representative Queries

Query 1: Total Sales by Product Category

```
SELECT
    p.Category,
    SUM(o.Quantity * o.Unit_Price) AS Total_Sales
FROM Orders o
JOIN OrderLines ol ON o.Order_ID = ol.Order_ID
JOIN Products p ON ol.Product_ID = p.Product_ID
GROUP BY p.Category;
```

In the star schema:

```
SELECT
    p.Category,
    SUM(f.Net_Sales) AS Total_Sales
FROM Fact_Sales f
JOIN Dim_Product p
ON f.Product_Key = p.Product_Key
GROUP BY p.Category;
```

Query 2: Monthly Sales by Store

```
SELECT
    d.Year,
    d.Month,
    s.Store_Name,
    SUM(f.Net_Sales) AS Total_Sales
FROM Fact_Sales f
JOIN Dim_Date d
ON f.Date_Key = d.Date_Key
JOIN Dim_Store s
ON f.Store_Key = s.Store_Key
GROUP BY d.Year, d.Month, s.Store_Name;
```

Query 3: Sales by Customer Segment

```
SELECT
    c.Customer_Segment,
    SUM(f.Net_Sales) AS Total_Sales
FROM Fact_Sales f
JOIN Dim_Customer c
ON f.Customer_Key = c.Customer_Key
GROUP BY c.Customer_Segment;
```

8.3 Performance Comparison

The following table provides the format for recording the experiment results.

Data Size	OLTP Time	Star Schema Time	Indexed Star Time
1,000 rows	Measure	Measure	Measure

10,000 rows	Measure	Measure	Measure
100,000 rows	Measure	Measure	Measure
1,000,000 rows	Measure	Measure	Measure

The execution times must be filled using actual database experiments. They should not be presented as measured results without running the queries.

8.4 Indexing

An index helps the database locate records faster without scanning the entire table.

Example:

```
CREATE INDEX idx_sales_date
ON Fact_Sales(Date_Key);


CREATE INDEX idx_sales_product
ON Fact_Sales(Product_Key);


CREATE INDEX idx_sales_store
ON Fact_Sales(Store_Key);


CREATE INDEX idx_sales_customer
ON Fact_Sales(Customer_Key);
```

These indexes can improve queries involving filtering and joining on dimension keys.

8.5 Aggregate Tables

An aggregate table stores precomputed summarized information.

Example:

```
CREATE TABLE Monthly_Store_Sales AS
SELECT
   d.Year,
   d.Month,
   f.Store_Key,
   SUM(f.Net_Sales) AS Total_Sales
FROM Fact_Sales f
JOIN Dim_Date d
ON f.Date_Key = d.Date_Key
```

```
GROUP BY
    d.Year,
    d.Month,
    f.Store_Key;
```

This table allows monthly store sales reports to retrieve precomputed totals rather than aggregate every individual sales record.

8.6 Query Plan and Performance Analysis

The following factors must be recorded for all three queries:

- Execution time.
- Number of rows scanned.
- Number of joins.
- Query execution plan.
- Additional index storage.
- Aggregate table storage.
- Data loading and refresh time.

An example of a query-plan inspection command in SQLite is:

```
EXPLAIN QUERY PLAN
SELECT
    Product_Key,
    SUM(Net_Sales)
FROM Fact_Sales
GROUP BY Product_Key;
```

8.7 Performance Discussion

The normalized OLTP database requires multiple joins because information is distributed across several tables.

The star schema simplifies analytical queries by connecting the fact table directly to dimension tables.

Indexes can improve data retrieval for selective filters and joins. Aggregate tables can improve frequently used summary reports.

However, indexes require additional storage and maintenance. Aggregate tables also require refreshing whenever new sales records are loaded.

Therefore, FreshCart should use indexing and selected aggregate tables rather than creating unnecessary structures.

8.8 Conclusion

The star schema with suitable indexes and aggregate tables is expected to provide better analytical performance for FreshCart. Actual execution measurements are necessary to validate the improvement.

TASK 9 – Build a KNIME Reporting Workflow (3 Marks)

9.1 Introduction

KNIME is an open-source data analytics platform used for data integration, data processing, visualization and reporting.

FreshCart can use KNIME to generate management reports from the data warehouse.

9.2 KNIME Workflow

Figure 9.1: KNIME Reporting Workflow

9.3 Nodes Used

KNIME Node	Purpose
CSV Reader	Reads warehouse data from CSV
DB Reader	Reads records from database
Missing Value	Handles missing values
Row Filter	Filters unwanted records
Joiner	Combines different tables
GroupBy	Calculates sales totals
Pivoting	Changes the report structure
Bar Chart	Displays sales comparisons
Table View	Displays detailed results

9.4 Workflow Explanation

Step 1 – Data Reading

Use the CSV Reader or DB Reader node to import Fact_Sales and dimension information.

Step 2 – Data Cleaning

Use Missing Value and Row Filter nodes to identify missing values and invalid records.

Step 3 – Data Integration

Use Joiner to combine sales information with Product, Store and Date dimensions.

Step 4 – Aggregation

Use GroupBy to calculate total sales by category, month or store.

Step 5 – Pivoting

Use Pivoting to create a cross-tab report comparing product categories across different stores.

Step 6 – Visualization

Use Bar Chart and Table View to generate management reports.

9.5 Mapping to ETL and OLAP

KNIME Component	Related Concept
CSV Reader	Extract
Missing Value	Transform
Row Filter	Data cleaning
Joiner	Data integration
GroupBy	Roll-up
Row Filter	Slice
Pivoting	Pivot
Bar Chart	Reporting

9.6 Limitation of KNIME

When data grows beyond one million rows, workflow execution may require more memory and processing time, depending on the machine and workflow design.

Large datasets may require database-side processing, optimized queries and sufficient system resources.

9.7 Conclusion

KNIME provides a visual and easy-to-understand method for building data processing workflows. It helps FreshCart generate reports without writing every operation manually.

TASK 10 – Recommend and Defend the Final Design (3 Marks)

10.1 Recommended Data Warehouse Design

After comparing the star schema, snowflake schema, ETL process and OLAP architecture, the recommended design for FreshCart is an Enterprise Data Warehouse using a star schema.

The system should contain:

- Centralized Enterprise Data Warehouse.
- Fact_Sales and Fact_Returns tables.
- Date, Product, Store, Customer and Promotion dimensions.
- ETL-based data integration.
- OLAP server for analytical operations.
- Dependent data marts for different departments.
- KNIME, Metabase or Apache Superset for reporting.

10.2 Reasons for Selecting the Design

1. Better Query Performance

The star schema requires fewer joins and supports efficient analytical queries.

2. Easy Data Understanding

The fact table is connected directly to dimension tables, making it easier for business users to understand.

3. Historical Data Analysis

The warehouse maintains historical information about products, stores, customers and promotions.

4. Data Consistency

A centralized warehouse ensures that all departments use the same definition of net sales.

5. Scalability

The design can support increasing sales records from thousands to millions.

10.3 Comparison of Alternatives

Design	Advantages	Disadvantages
Independent Data Marts	Easy to build separately	Data inconsistency
Virtual Warehouse	Less physical storage	Depends on source systems
Snowflake Schema	Less redundant dimension data	More joins

Enterprise Star Schema	Simple queries and centralized reporting	Some dimension redundancy

10.4 Trade-Offs and Limitations

The recommended design requires additional storage because historical data is maintained separately.

ETL processing also introduces additional maintenance work.

Indexes and aggregate tables improve reporting but increase storage requirements and loading costs.

These costs are acceptable because the warehouse improves management reporting and protects the operational system from complex analytical workloads.

10.5 Scalability

The proposed architecture can support more than one million fact records by using:

- Appropriate indexes.
- Incremental ETL loading.
- Partitioning large fact tables when supported.
- Aggregate tables.
- Database query optimization.
- Efficient OLAP processing.

10.6 Conclusion

The Enterprise Data Warehouse with a star schema is the most suitable choice for FreshCart. It provides a balance between query performance, scalability, data consistency and ease of reporting.

TASK 11 – Re-evaluate and Adapt the Design (Day-4 Design Challenge – 8 Marks)

11.1 Introduction

FreshCart introduces a new online sales channel that generates approximately 20,000 orders per day.

The new system introduces additional requirements:

1. Different product code formats.
2. A new Delivery Partner attribute.
3. Frequent changes in customer addresses.
4. Frequent changes in customer loyalty tiers.

5. Dashboards must be refreshed every hour.

These changes require FreshCart to evaluate its existing warehouse architecture and modify the ETL process.

A. Impact Analysis (2 Marks)

The new online sales channel affects several parts of the existing warehouse.

Component	Impact	Required Change
Data Sources	High	Integrate online sales system
Fact_Sales	Medium	Include online sales transactions
Dim_Product	High	Support different product codes
Dim_Customer	High	Maintain address and loyalty history
Dim_Store	Low	Retain existing structure
ETL Process	High	Support multiple source formats
Data Cube	Medium	Add online channel analysis
OLAP Server	Medium	Support additional dimensions
Reporting	High	Refresh dashboards hourly

Explanation

The online channel introduces new data formats and attributes. Therefore, the existing product dimension must support product code mapping from different systems.

Customer information must be maintained historically because management wants to analyse sales using the address and loyalty tier applicable at the time of the transaction.

The ETL process must also support hourly updates instead of relying only on large periodic loads.

B. Evaluation of the Original Design (2 Marks)

The original design is suitable for the existing 120 supermarkets. However, the new online sales channel increases the data volume.

Additional Data Volume

Online orders per day:

\[20,000 \]

Online orders per year:

\[20,000\times365=7,300,000 \]

7.3 million online orders per year

If each order contains an average of three product line items:

\[20,000\times3=60,000 \]

Daily online sales line items:

\[60,000 \]

Annual online sales line items:

\[60,000\times365=21,900,000 \]

This shows that the warehouse must be capable of handling a significantly larger volume of sales data.

Problems with the Original Design

1. The original ETL process may not support hourly refreshes.
2. Different product codes may cause duplicate product records.
3. Customer changes may overwrite historical information.
4. New delivery partner information is not available in the original model.
5. Large data volumes may increase query processing time.

Therefore, the original design should be modified rather than completely replaced.

C. Adaptation or Redesign (2 Marks)

The recommended solution is to retain the Enterprise Data Warehouse and star schema while introducing additional dimensions and an improved ETL process.

1. Add Dim_Sales_Channel

Attribute	Description
Channel_Key	Primary key
Channel_ID	Channel identifier
Channel_Name	Store or Online
Channel_Type	Sales channel

This dimension allows management to compare physical supermarket sales and online sales.

2. Add Dim_Delivery_Partner

Attribute	Description
Delivery_Key	Primary key
Partner_ID	Partner identifier
Partner_Name	Delivery company
Partner_Type	Delivery category

This dimension supports delivery partner performance analysis.

3. Modify Fact_Sales

The Fact_Sales table will include two additional foreign keys:

- Channel_Key
- Delivery_Key

The Delivery_Key can use an Unknown or Not Applicable member for physical store transactions.

4. Use SCD Type 2

Customer address and loyalty tier changes will be maintained using SCD Type 2.

This ensures that historical sales records remain connected to the correct customer information.

5. Improve ETL Using Micro-Batch Processing

Instead of processing all records at the end of the day, FreshCart should use hourly incremental ETL.

Proposed process:

Figure 11.1: Revised FreshCart Architecture

The process includes:

1. Collect online and supermarket data.
2. Extract new and modified records.
3. Store data in the staging area.
4. Standardize product codes and customer information.
5. Apply SCD Type 2.
6. Load dimension tables.
7. Load new sales facts.

8. Refresh selected aggregate tables.
9. Update management dashboards.

D. Evidence-Based Defence (2 Marks)

The revised design is more suitable because it supports the new online channel without replacing the entire warehouse.

Design Comparison

Requirement	Original Design	Revised Design
Supermarket sales	Supported	Supported
Online sales	Not included	Supported
Multiple product codes	Limited	Standardized
Delivery partner analysis	Not supported	Supported
Historical customer changes	Requires enhancement	SCD Type 2
Hourly refresh	Not designed for it	Micro-batch ETL
Large data volume	Requires testing	Designed to scale

Performance Evaluation

The revised system should be tested using the following measurements:

Parameter	Measurement
Daily online orders	20,000
Daily online line items	60,000 (assuming 3 per order)
Hourly order volume	Approximately 833
Annual online orders	7,300,000
Dashboard refresh interval	1 hour
ETL processing time	Measure experimentally
Query execution time	Measure experimentally

The hourly order volume is calculated as:

$$\frac{20,000}{24}\approx833$$

This is an average. Actual hourly traffic may be higher during peak periods.

Monitoring After Deployment

FreshCart should continuously monitor:

- ETL execution time.
- Data loading failures.
- Duplicate product records.
- Missing customer information.
- Dashboard refresh delays.
- Query execution time.
- Warehouse storage growth.
- Data reconciliation errors.

11.2 Final Conclusion

The original Enterprise Data Warehouse design is suitable for FreshCart's supermarket operations. However, the introduction of online sales requires additional dimensions, improved data integration and hourly ETL processing.

The recommended approach is to retain the existing star schema and modify it by introducing Sales Channel and Delivery Partner dimensions. SCD Type 2 should be used to preserve customer history.

The revised architecture provides better scalability, historical accuracy and reporting capabilities. It also supports FreshCart's future business expansion.

OVERALL ASSIGNMENT CONCLUSION

The FreshCart Retail Analytics project demonstrates how a data warehouse can improve business intelligence and decision-making in a large retail organization.

The existing OLTP database is useful for daily transactions but is not suitable for complex analytical queries. Therefore, an Enterprise Data Warehouse has been proposed to integrate data from all supermarkets and online sales channels.

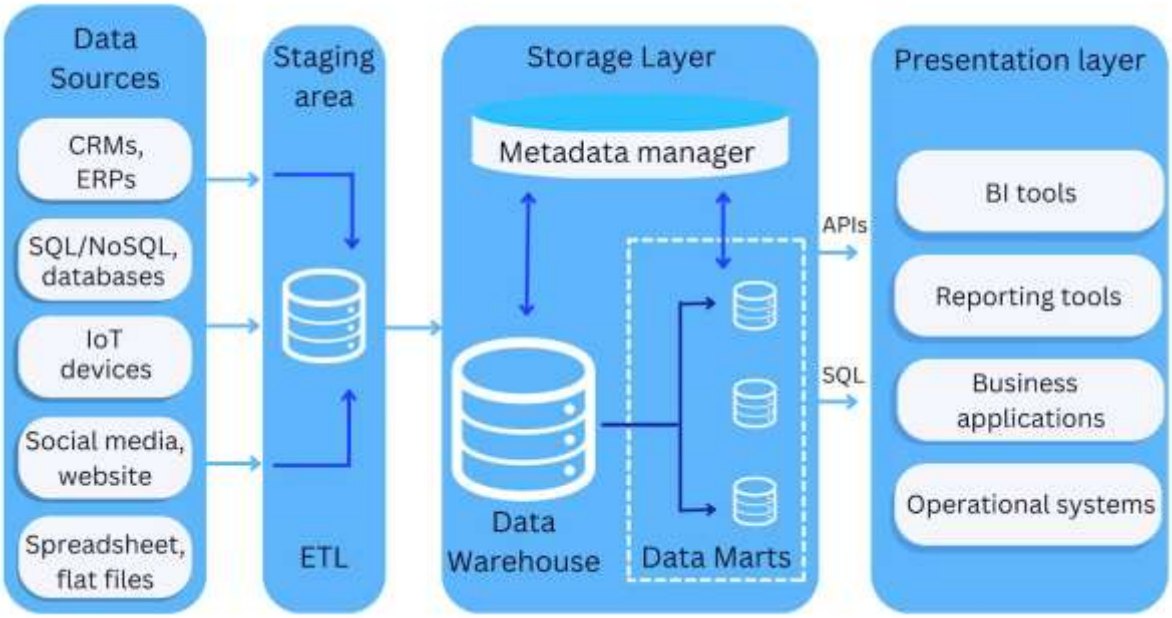
A star schema was selected because it provides a simple structure, reduces the number of joins and supports efficient analytical queries. The snowflake schema was also examined to understand its storage benefits and additional join requirements.

The ETL process ensures that data from different sources is extracted, cleaned, transformed and loaded correctly. Surrogate keys and Slowly Changing Dimension Type 2 help maintain accurate relationships and historical customer information.

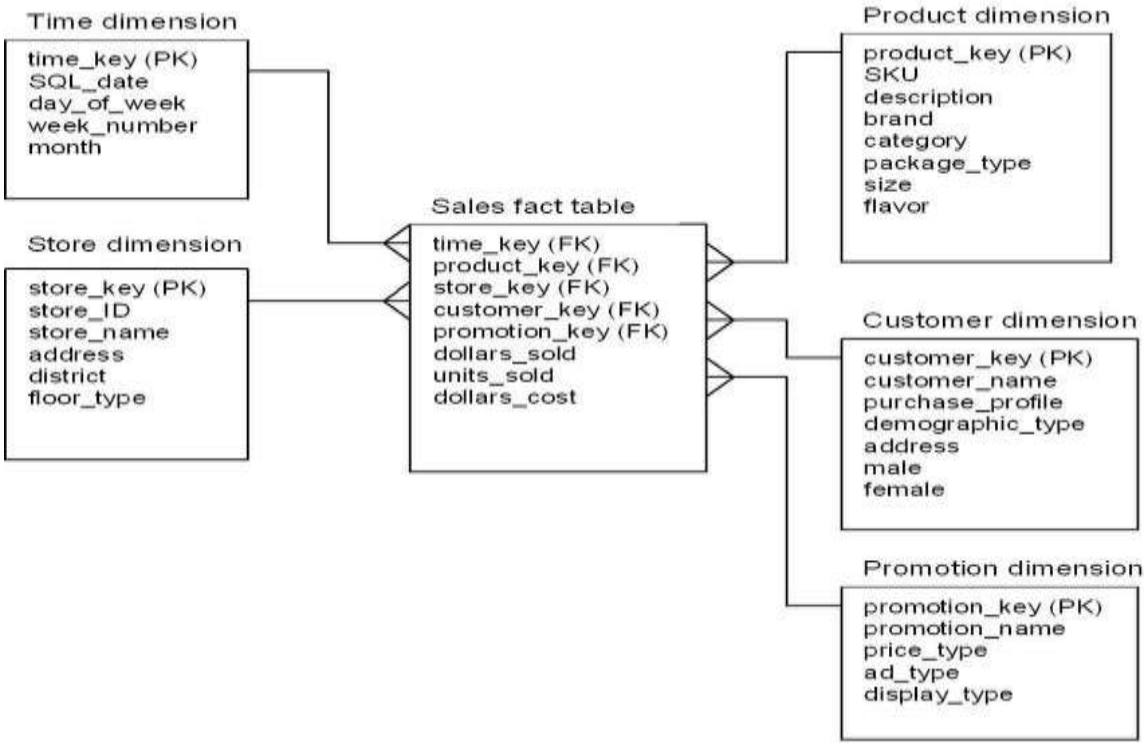
Data cubes and OLAP operations such as roll-up, drill-down, slice, dice and pivot allow management to analyse sales from different perspectives. Python, SQL and KNIME provide practical methods for developing analytical applications and management reports.

Finally, the design challenge demonstrated how the warehouse can be extended to support online sales, delivery partner information and hourly dashboard updates.

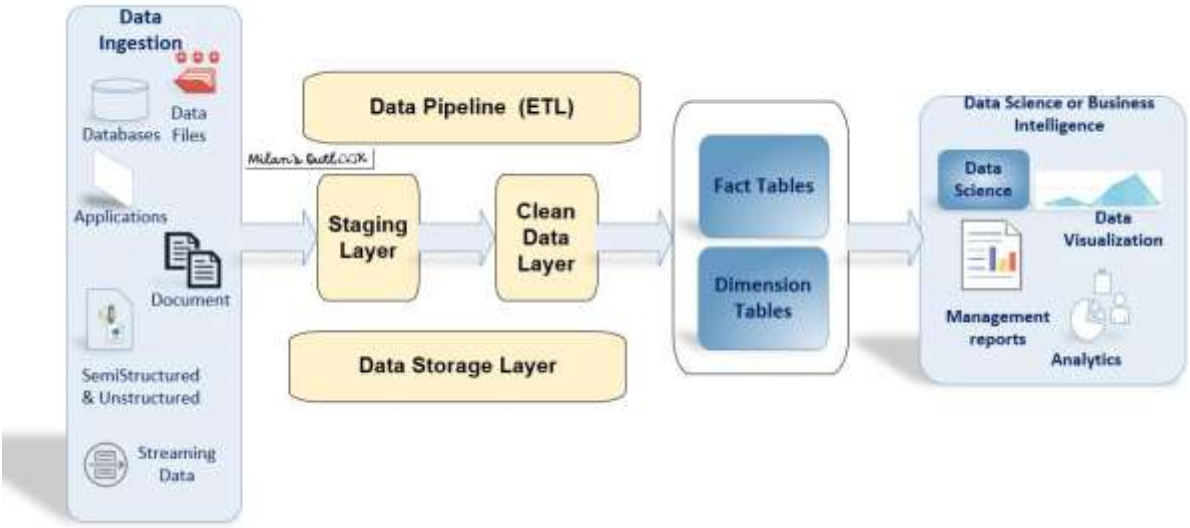
Thus, the proposed FreshCart Data Warehouse provides a scalable and reliable solution for retail data management, sales analysis and business decision-making.



2.2


Time dimension
time_key (PK)
SQL_date
day_of_week
week_number
month
Store dimension
store_key (PK)
store_ID
store_name
address
district
floor_type
Sales fact table
time_key (FK)
product_key (FK)
store_key (FK)
customer_key (FK)
promotion_key (FK)
dollars_sold
units_sold
dollars_cost
Product dimension
product_key (PK)
SKU
description
brand
category
package_type
size
flavor
Customer dimension
customer_key (PK)
customer_name
purchase_profile
demographic_type
address
male
female
Promotion dimension
promotion_key (PK)
promotion_name
price_type
ad_type
display_type


SNOWFLAKE SCHEMA: PRODUCT SALES
MONTH DIMENSION
YEAR DIMENSION
DATE DIMENSION
CATEGORY DIMENSION
REGION DIMENSION
PRODUCT DIMENSION
SALES FACT TABLE
SALESPERSON DIMENSION
BRAND DIMENSION
COMMISSION RATE DIMENSION
CUSTOMER DIMENSION
PROMOTION DIMENSION
INDUSRTY DIMENSION
COMPANY SIZE DIMENSION
PROMOTION TYPE DIMENSION
CHANNEL DIMENSION

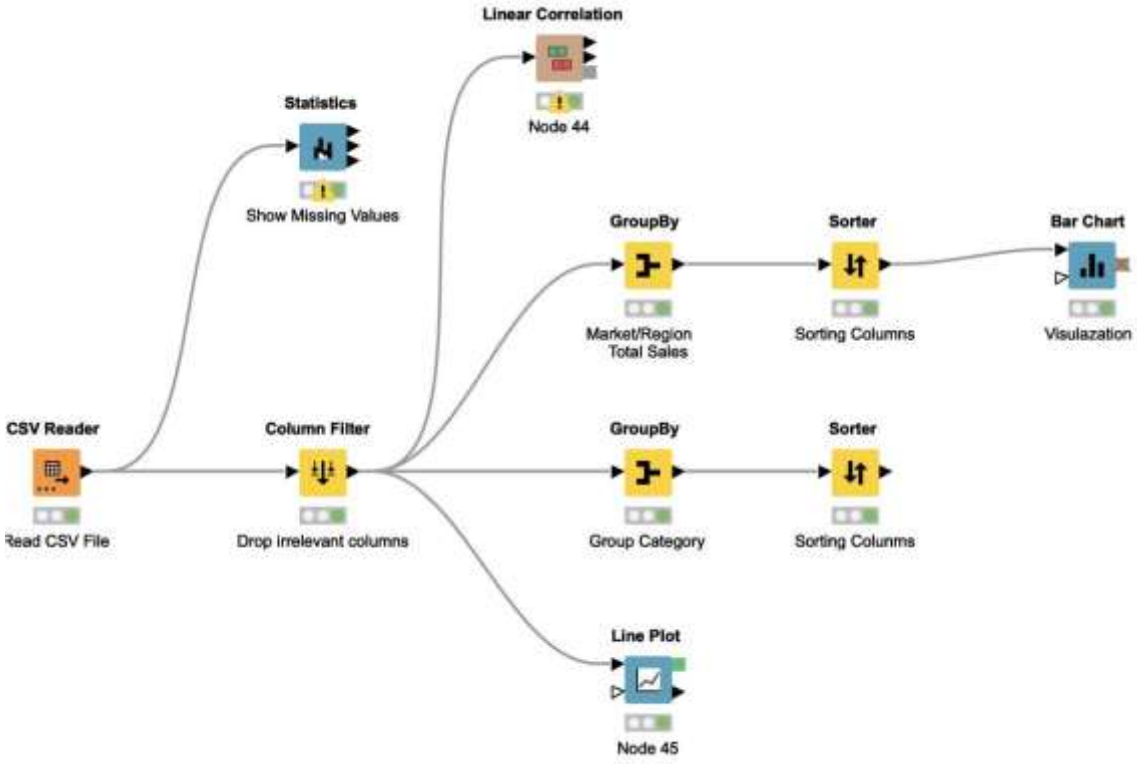


Operations on OLAP Cube

Illustrations of Dicing, Drill-down, Roll-up, Slicing, Pivot



Get these slides & Icons at www.InfoDiagram.com


Linear Correlation
Node 44
Statistics
Show Missing Values
GroupBy
Market/Region
Total Sales
Sorter
Sorting Columns
Bar Chart
Visulazation
CSV Reader
Read CSV File
Column Filter
Drop irrelevant columns
GroupBy
Group Category
Sorter
Sorting Columns
Line Plot
Node 45